

Name: Monisha Raghini Chelle

RollNo: 2520030537

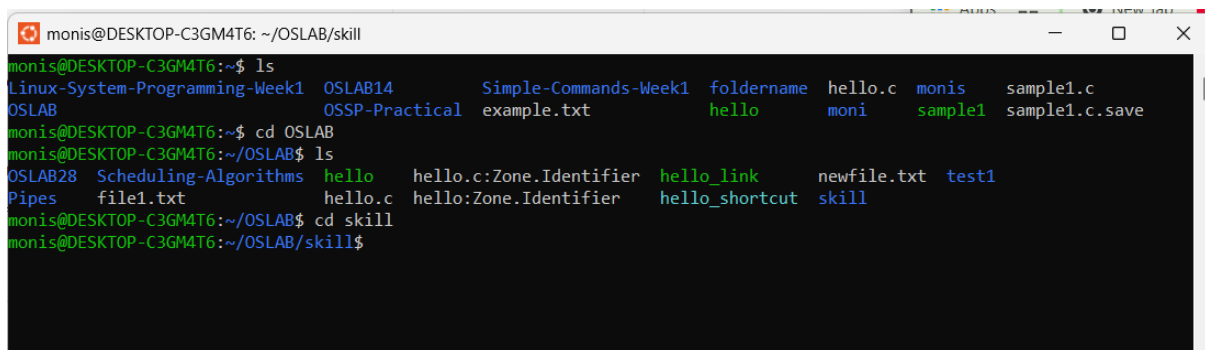
Section-6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

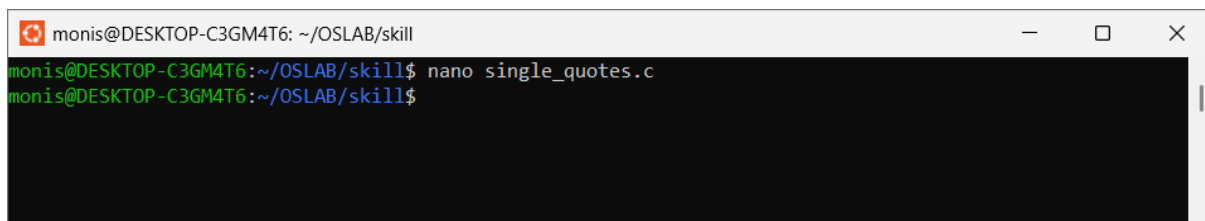
5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14  Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                          OSSP-Practical  example.txt             hello       moni     sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the Single-Quote Parser



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano single_quotes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 single_quotes.c *
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_OUTPUT 200

void parse_single_quotes(char *input, char *output)
{
    int i = 0;
    int j = 0;
    int in_single_quote = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '\')
        {
            in_single_quote = !in_single_quote;
            i++;
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';

    if (in_single_quote)
    {
        printf("Error: Unmatched single quote.\n");
    }
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 single_quotes.c *
int j = 0;
int in_single_quote = 0;

while (input[i] != '\0' && j < MAX_OUTPUT - 1)
{
    if (input[i] == '\'' )
    {
        in_single_quote = !in_single_quote;
        i++;
    }
    else
    {
        output[j++] = input[i++];
    }
}

output[j] = '\0';

if (in_single_quote)
{
    printf("Error: Unmatched single quote.\n");
}
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    parse_single_quotes(input, output);

    printf("\nParsed output: [%s]\n", output);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^V Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

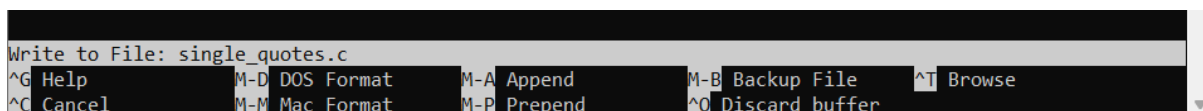
Save

Press:

Ctrl + O

Enter

Ctrl + X



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
Write to File: single_quotes.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel     M-M Mac Format  M-P Prepend    ^Q Discard buffer
```

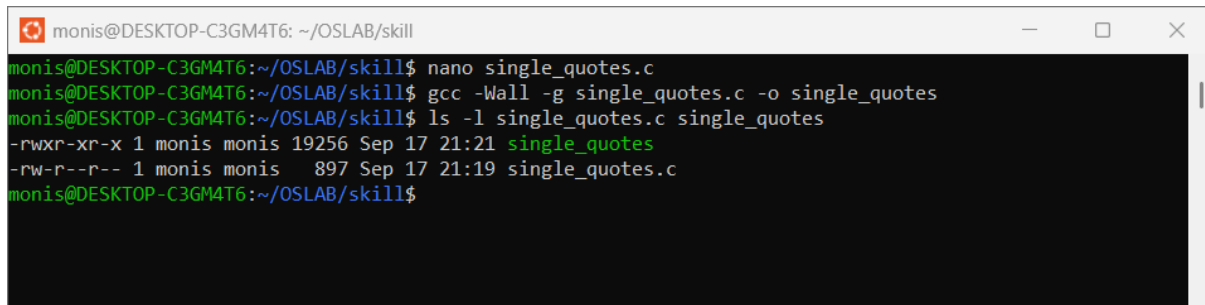
Step 3 — Compile the Single-Quote Parser

Run:

```
gcc -Wall -g single_quotes.c -o single_quotes
```

If there are no errors, check that the source file and executable exist:

```
ls -l single_quotes.c single_quotes
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano single_quotes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g single_quotes.c -o single_quotes
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l single_quotes.c single_quotes
-rwxr-xr-x 1 monis monis 19256 Sep 17 21:21 single_quotes
-rw-r--r-- 1 monis monis  897 Sep 17 21:19 single_quotes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Test Single Quotes

Now run:

```
./single_quotes
```

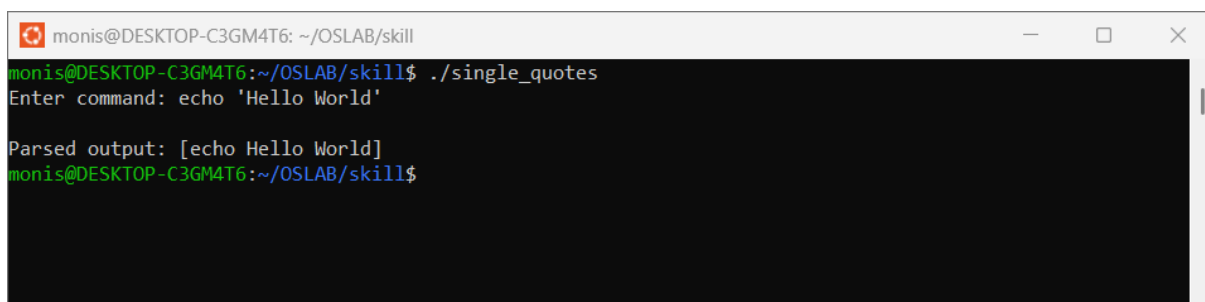
At:

Enter command:

type exactly:

```
echo 'Hello World'
```

Press Enter.



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./single_quotes
Enter command: echo 'Hello World'

Parsed output: [echo Hello World]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Test Variable Expansion Inside Single Quotes

Run:

```
./single_quotes
```

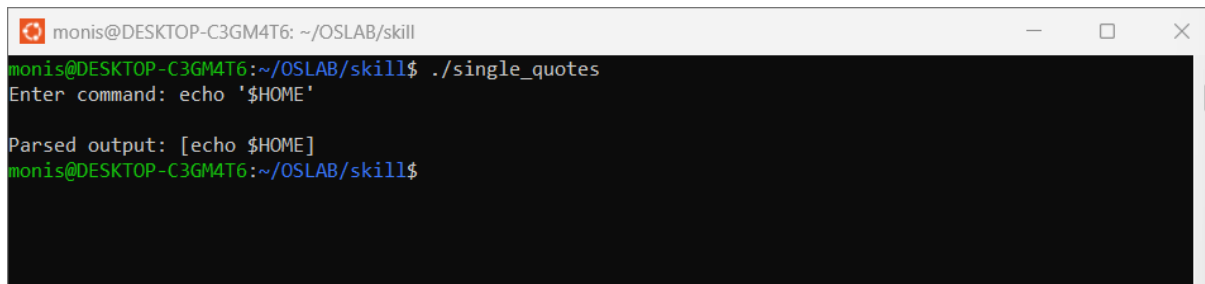
At:

Enter command:

type exactly:

echo '\$HOME'

Press Enter.



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./single_quotes
Enter command: echo '$HOME'

Parsed output: [echo $HOME]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 6 — Test Single-Quote Edge Case

Run:

./single_quotes

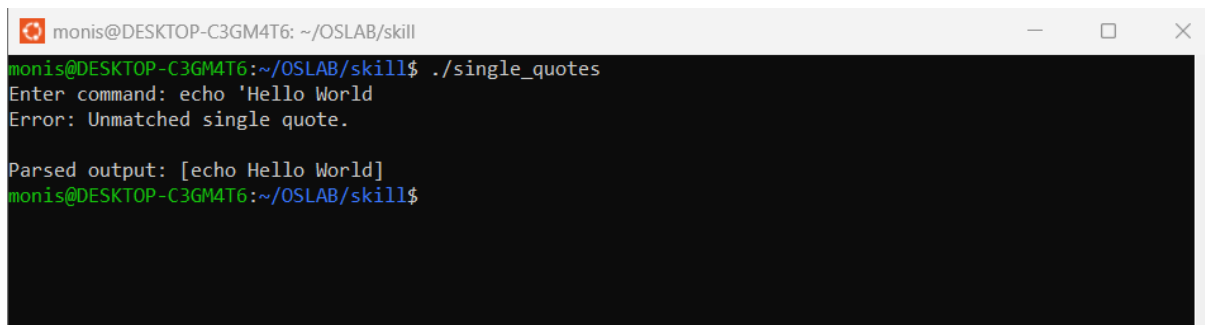
At:

Enter command:

type exactly:

echo 'Hello World

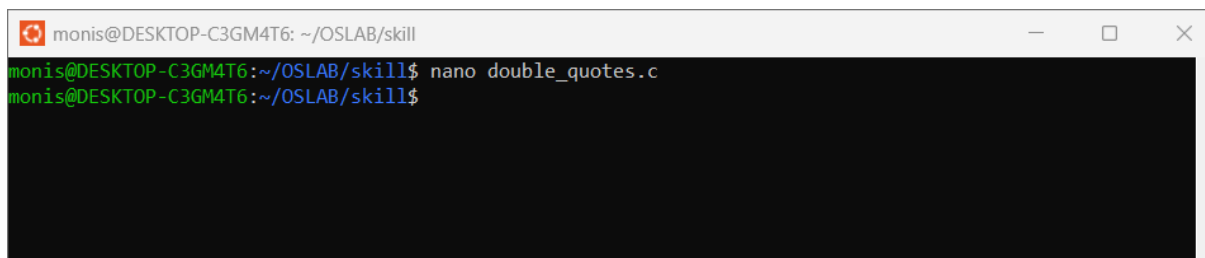
Press Enter.



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./single_quotes
Enter command: echo 'Hello World
Error: Unmatched single quote.

Parsed output: [echo Hello World]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 7 — Create the Double-Quote Parser



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano double_quotes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 double_quotes.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_OUTPUT 400

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;
            char *value;

            i++;

            while (input[i] != '\0' &&
                ((input[i] >= 'A' && input[i] <= 'Z') ||
                 (input[i] >= 'a' && input[i] <= 'z') ||
                 (input[i] >= '0' && input[i] <= '9') ||
                 input[i] == '_'))
            {
                if (k < 99)
                {
                    variable[k++] = input[i];
                }

                i++;
            }

            variable[k] = '\0';

            value = getenv(variable);

            if (value != NULL)
            {
                int len = strlen(value);

                if (j + len < MAX_OUTPUT - 1)

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 double_quotes.c *
    if (j + len < MAX_OUTPUT - 1)
    {
        strcpy(&output[j], value);
        j += len;
    }
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';

void parse_double_quotes(char *input, char *output)

int i = 0;
int j = 0;
int in_double_quote = 0;

while (input[i] != '\0' && j < MAX_OUTPUT - 1)
{
    if (input[i] == '"')
    {
        in_double_quote = !in_double_quote;
        i++;
    }
    else
    {
        output[j++] = input[i++];
    }
}

output[j] = '\0';

if (in_double_quote)
{
    printf("Error: Unmatched double quote.\n");
    return;
}

char expanded[MAX_OUTPUT];

G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 double_quotes.c *
    in_double_quote = !in_double_quote;
    i++;
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';

if (in_double_quote)
{
    printf("Error: Unmatched double quote.\n");
    return;
}

char expanded[MAX_OUTPUT];

expand_variables(output, expanded);

strcpy(output, expanded);
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    parse_double_quotes(input, output);

    printf("\nParsed output: [%s]\n", output);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

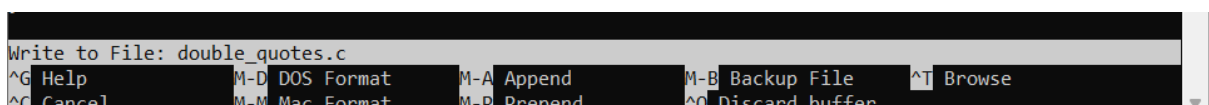
Save the file

Press:

Ctrl + O

Enter

Ctrl + X



```
Write to File: double_quotes.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Step 8 — Compile the Double-Quote Parser

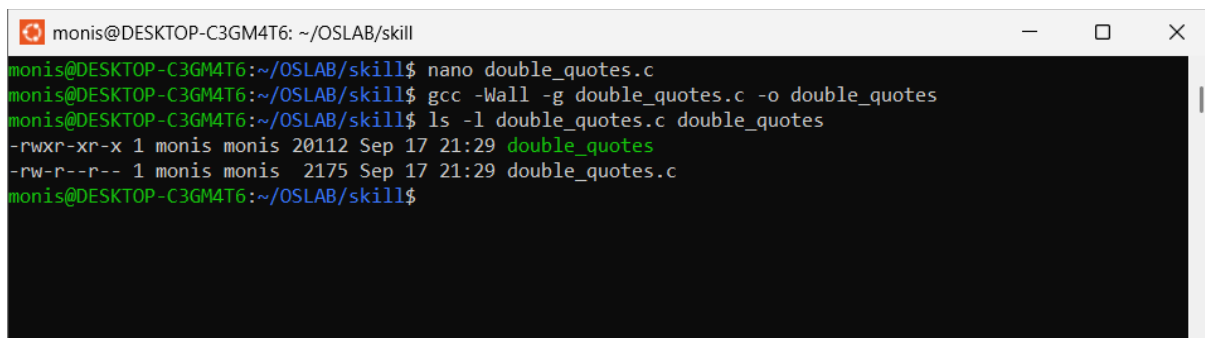
Run:

```
gcc -Wall -g double_quotes.c -o double_quotes
```

If there are no errors, check that the source file and executable exist:

```
ls -l double_quotes.c double_quotes
```

[You should see both files listed.]



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano double_quotes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g double_quotes.c -o double_quotes
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l double_quotes.c double_quotes
-rwxr-xr-x 1 monis monis 20112 Sep 17 21:29 double_quotes
-rw-r--r-- 1 monis monis 2175 Sep 17 21:29 double_quotes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 9 — Test Double Quotes with Spaces

Run:

```
./double_quotes
```

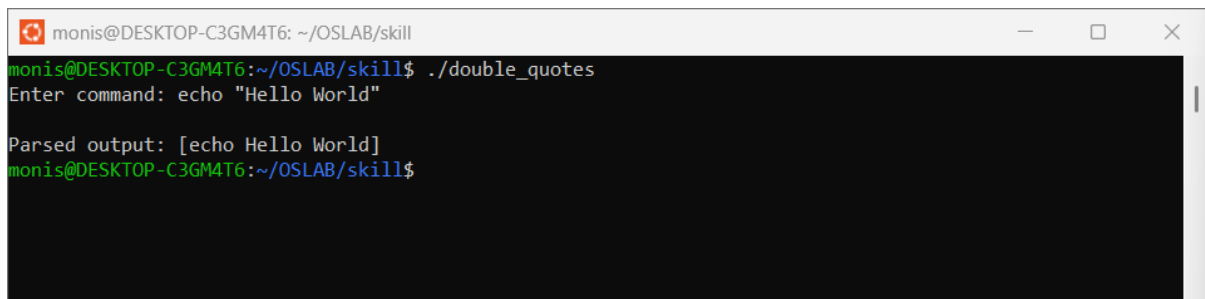
At:

Enter command:

type exactly:

```
echo "Hello World"
```

Press Enter.



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./double_quotes
Enter command: echo "Hello World"

Parsed output: [echo Hello World]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 10 — Test Variable Expansion Inside Double Quotes

Run:

`./double_quotes`

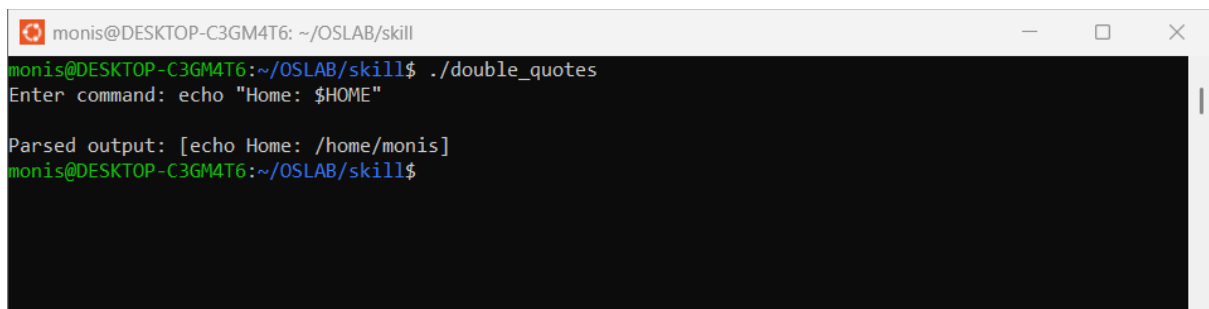
At:

Enter command:

type exactly:

`echo "Home: $HOME"`

Press Enter.

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'. The user enters './double_quotes'. The prompt changes to 'Enter command:'. The user enters 'echo "Home: \$HOME"'. The prompt changes to 'Parsed output:'. The output is '[echo Home: /home/monis]'. The prompt returns to 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'.

Step 11 — Test a Quoted Command with Multiple Tokens

Run:

`./double_quotes`

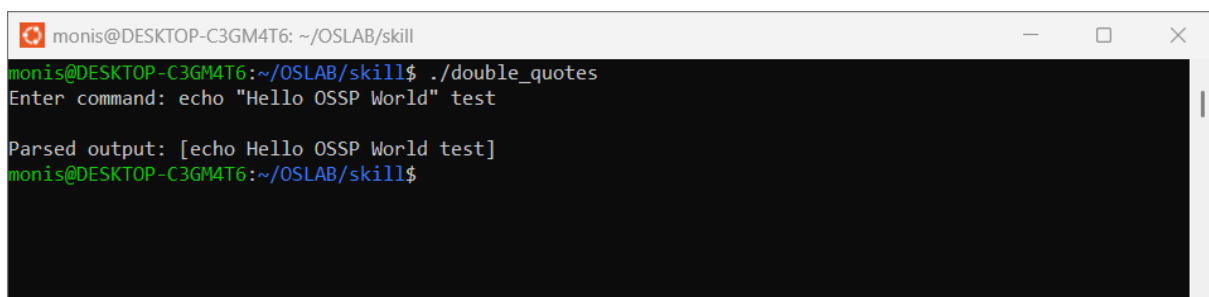
At:

Enter command:

type exactly:

`echo "Hello OSSP World" test`

Press Enter.

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'. The user enters './double_quotes'. The prompt changes to 'Enter command:'. The user enters 'echo "Hello OSSP World" test'. The prompt changes to 'Parsed output:'. The output is '[echo Hello OSSP World test]'. The prompt returns to 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'.

Step 12 — Test Unmatched Double Quotes

Run:

`./double_quotes`

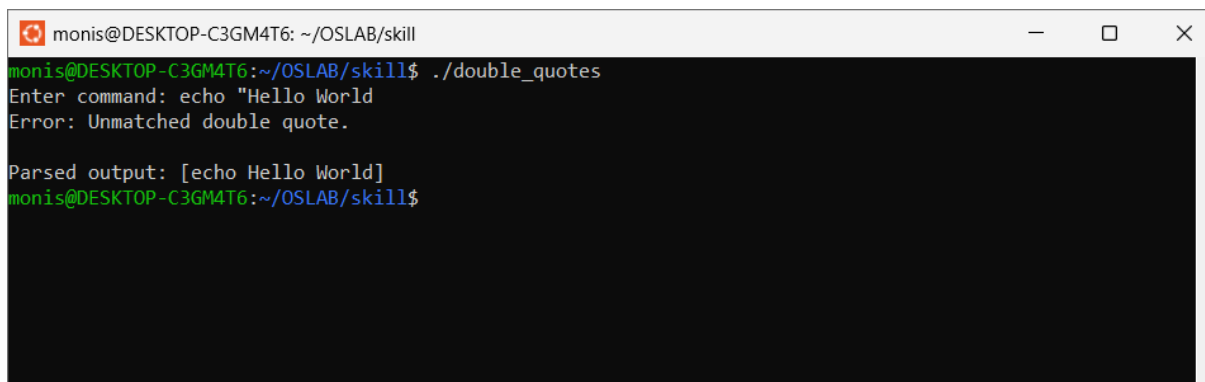
At:

Enter command:

type exactly:

`echo "Hello World"`

Press Enter.

A terminal window titled "monis@DESKTOP-C3GM4T6: ~/OSLAB/skill". The prompt is "monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$". The user enters the command `./double_quotes`. The terminal displays "Enter command: echo \"Hello World\"", followed by an error message "Error: Unmatched double quote." and then "Parsed output: [echo Hello World]". The prompt returns to "monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$".

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./double_quotes
Enter command: echo "Hello World"
Error: Unmatched double quote.
Parsed output: [echo Hello World]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 13 — Test a Complex Double-Quoted Command

Run:

`./double_quotes`

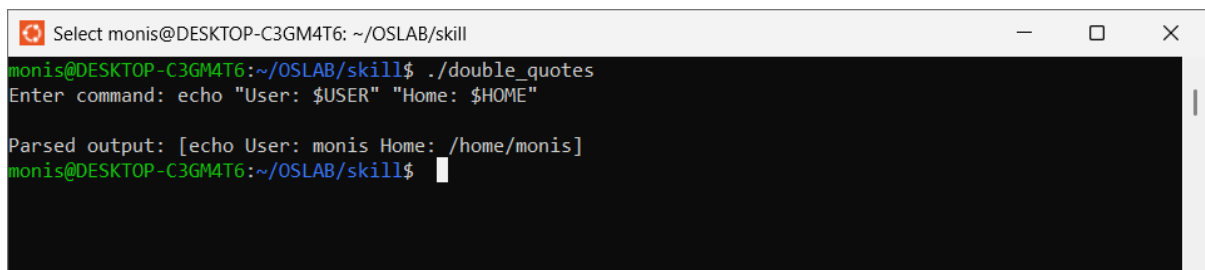
At:

Enter command:

type exactly:

`echo "User: $USER" "Home: $HOME"`

Press Enter.

A terminal window titled "Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill". The prompt is "monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$". The user enters the command `./double_quotes`. The terminal displays "Enter command: echo \"User: \$USER\" \"Home: \$HOME\"", followed by "Parsed output: [echo User: monis Home: /home/monis]". The prompt returns to "monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$".

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./double_quotes
Enter command: echo "User: $USER" "Home: $HOME"
Parsed output: [echo User: monis Home: /home/monis]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 14 — Final Q5 Verification

Run:

```
./single_quotes
```

Enter:

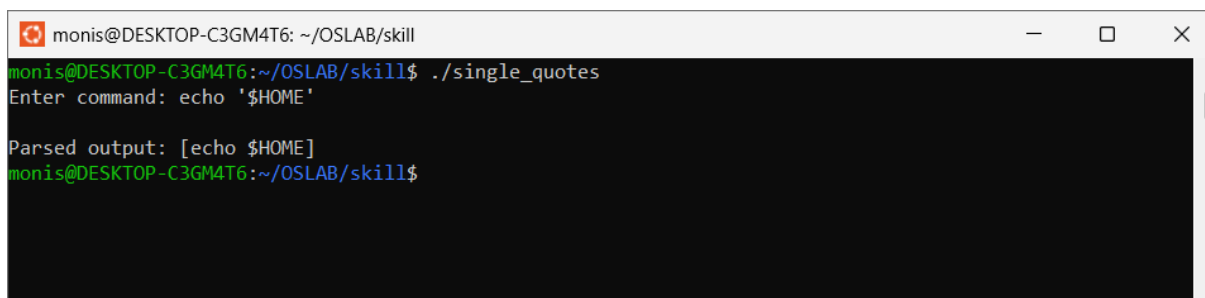
```
echo '$HOME'
```

Then run:

```
./double_quotes
```

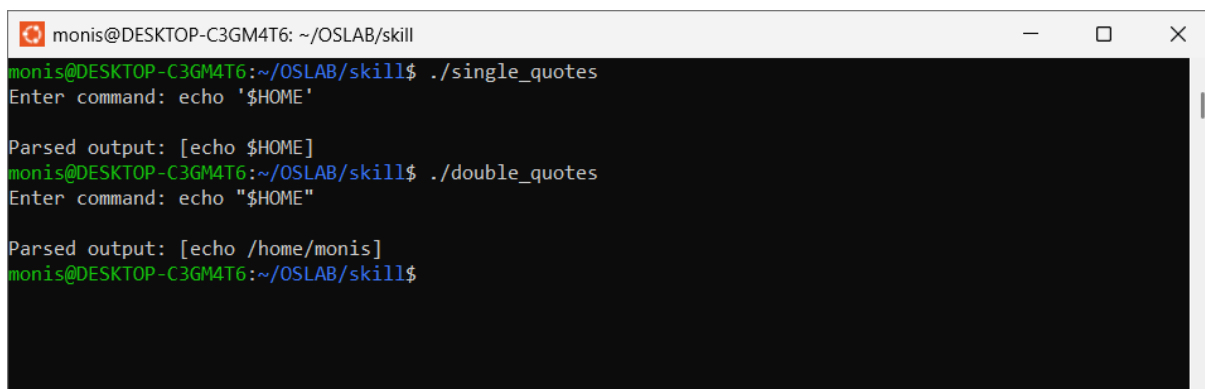
Enter:

```
echo "$HOME"
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./single_quotes
Enter command: echo '$HOME'

Parsed output: [echo $HOME]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./single_quotes
Enter command: echo '$HOME'

Parsed output: [echo $HOME]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./double_quotes
Enter command: echo "$HOME"

Parsed output: [echo /home/monis]
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

